

MetroLift's Last Capstone Review

Table of Contents

1. [Learning Objectives](#)
2. [Case Pipeline](#)
3. [Friday, 16:40: the demo survived, the review did not](#)
4. [What the team learned before the room got tense](#)
5. [Exhibits from the platform review pack](#)
6. [Questions the review panel wants answered](#)
7. [Model answers Priya wished the team had rehearsed](#)
8. [How the platform should run after the applause](#)
9. [Transfer the lesson beyond MetroLift](#)
10. [Decision Points](#)
11. [Analysis Walkthrough](#)

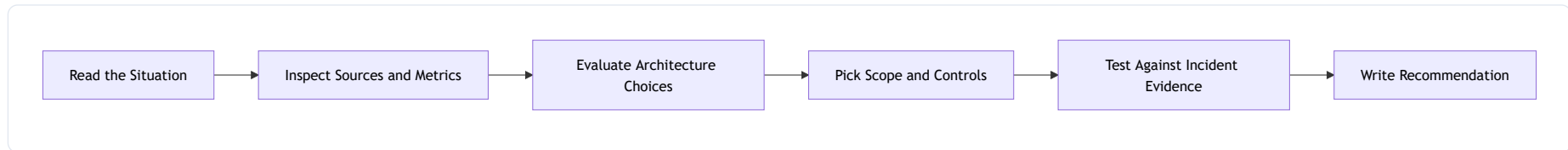
 Duration: 60 minutes

Learning Objectives

- Define a credible AI-native platform scope from an ambiguous operations-assistant request.
- Choose an architecture, transformation path, and AI-native component that match source realities and consumer metrics.
- Defend governance, quality, CI/CD, and observability controls using a risk-based operating model.

- Compare recovery and release options using concrete evidence from pipeline, retrieval, and data-quality exhibits.

Case Pipeline



Friday, 16:40: the demo survived, the review did not

Sep 27, 2025. MetroLift’s capstone team is in a glass room on the 11th floor, thirty minutes after a practice demo looked fine. Then Priya Nair, the staff data engineer leading the build, gets the question that matters: **“Can another engineer look at your brief, your sources, your controls, and infer exactly why this platform is shaped this way?”** The room goes quiet because the answer is only half yes.

MetroLift maintains commercial elevators across six cities. The business pain is specific: technicians are arriving on-site with fragmented context, spending 12-18 minutes hunting through PDFs and ticket history, and logging repeat visits on common controller faults at **18.4%**. The proposed product is a technician-facing assistant that answers repair questions with citations, plus a curated data product for service planners.

What is at stake is not just the final grade. Operations committed a **2025-Q4 pilot across 220 technicians** if the capstone shows credible controls. A sloppy build would push technicians toward ungrounded answers, expose customer data from raw attachments, and hand operations a system nobody can recover when the nightly run breaks.

Priya needs a recommendation by the end of the review on four linked decisions: **scope, base architecture, AI-native component, and operating controls**. The learner steps into that review.

What the team learned before the room got tense

- **2025-07-08:** Operations PM Lena Ortiz writes the initial brief as “AI assistant for field service efficiency,” with no named consumers, no source inventory, and no success targets beyond “faster answers.”
- **2025-07-15:** Priya rewrites the brief into a contract test: primary consumer = field technician; secondary consumers = service planners and compliance reviewers; inputs = work orders, parts inventory, approved manuals, service bulletins; outputs = cited answer API and curated service-event product.
- **2025-07-24:** Source assessment flags four approved sources and two risky ones. Approved: SQL Server work orders, nightly CSV inventory feed, SharePoint manuals, bulletin PDFs. Risky: voice-note transcripts with low completeness and warranty attachments containing PII.
- **2025-08-05:** Architecture spike compares warehouse-first and lakehouse-first patterns. Lakehouse-first wins on raw landing, document lineage, and replayability, but only if the team limits scope to one assistant and one curated product.
- **2025-08-21:** The team implements Python ingestion, Spark text normalization for PDFs, dbt models for curated service facts, and a FastAPI stub for retrieval and answer serving.
- **2025-09-11:** CI/CD gates are added for dbt tests, schema checks, manifest completeness, chunk-count deltas, retrieval eval drift, and PII scans on AI-ready text.
- **2025-09-18:** A failed bulletin load publishes an incomplete index anyway. The issue is rolled back manually in 94 minutes, exposing a missing promotion gate between corpus build and embedding release.

The scope boundary is now written down, which is why the team still has a shot. **In scope:** daily ingestion, one trusted curated data product, one governed corpus, one RAG API, evaluation set, lineage docs, and runbook. **Deferred:** agentic workflows, production voice-note search, model fine-tuning, automated dispatch actions. **Out of scope:** customer-facing chatbot and workflow execution against ERP.

Priya’s risk register has three items circled in red:

Risk	Likelihood	Impact	Mitigation
Incomplete document ingest corrupts retrieval relevance	Medium	High	Manifest gate, corpus versioning, index promotion only after completeness checks

Sensitive text leaks from raw attachments into AI-ready corpus	Medium	High	Restrict raw zone, source approval list, PII scan before curated AI release
Scope creep from assistant to agent breaks delivery	High	High	ADR locking consumer, output contract, and deferred action paths

Exhibits from the platform review pack

The review packet combines source reality, consumer metrics, and the failure that almost sank trust.

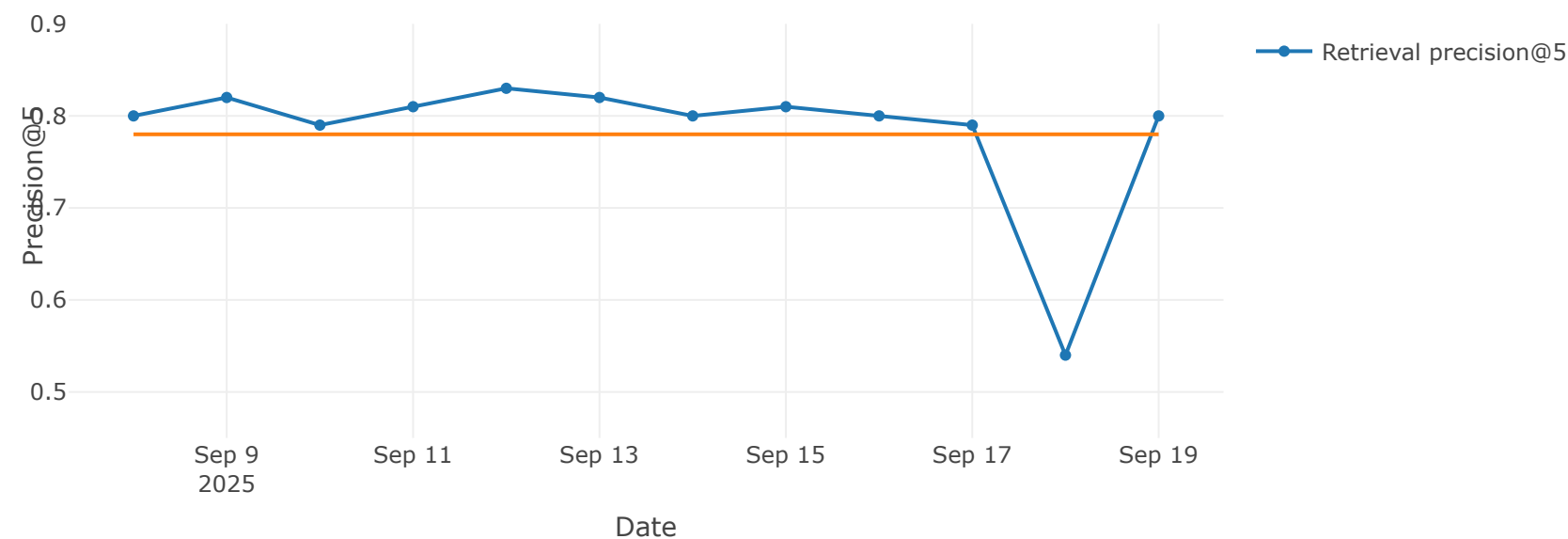
Source	Format	Daily freshness need	Owner	Sensitivity	Quality signal	Recommendation
Work orders	SQL Server tables	24 h	Service Ops	Medium	99.2% required fields present; status taxonomy stable	Usable
Parts inventory	CSV export	24 h	ERP team	Low	3.4% SKU alias mismatch across regions	Usable with controls
Service manuals	PDF	Weekly	Engineering Docs	Low	OCR clean on 96% of pages	Usable
Service bulletins	PDF	24 h	Reliability Eng	Low	Late postings 2-3 times/month	Usable with controls
Voice-note	JSON text	24 h	Mobile Apps	Medium	61% complete;	Deferred

transcripts					terminology drift severe	
Warranty attachments	Email/PDF	48 h	Claims	High	Free-text PII common; no owner SLA	Out of scope

Consumer-centered targets are intentionally small.

Metric type	Metric	Target	Owner	Evidence source
Business	Repeat-visit rate on pilot fault classes	<= 15.0% within 90 days	Ops PM	Service KPI dashboard
Business	Technician prep time before job	<= 5 min median	Regional managers	Time-and-motion sample
AI	Citation coverage	100% of answers	Priya	API response logs
AI	Grounded answer pass rate	>= 85% on 120-question eval set	ML engineer	Versioned eval runs
AI	Retrieval precision@5	>= 0.78	ML engineer	Retrieval eval harness
Platform	API latency	p95 < 8 s	Platform eng	FastAPI telemetry
Platform	Curated freshness	< 24 h	Data eng	Orchestrator run metadata
Platform	Failed-run recovery	< 2 h	On-call owner	Incident records

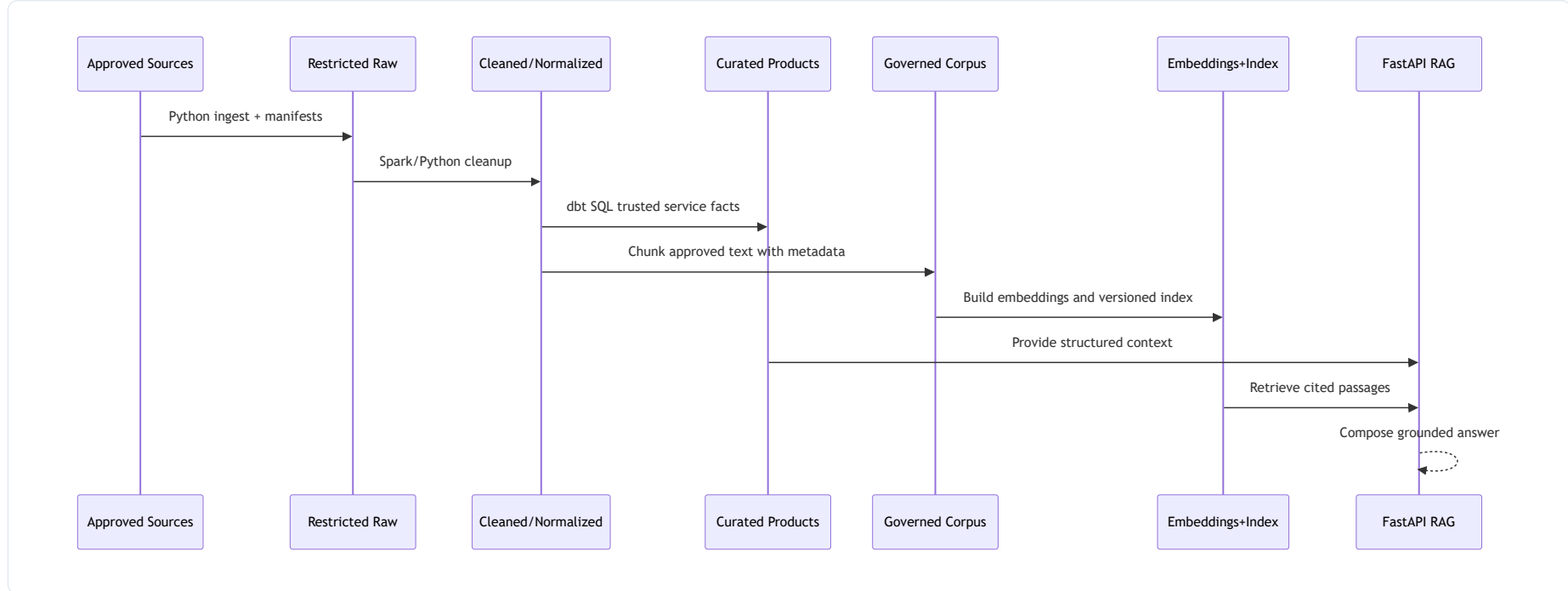
Retrieval precision@5 around the bulletin ingest incident



The failed-run exhibit shows why “pipeline success” alone is a bad metric.

Run date	Expected bulletins	Landed	Chunk records	Index version published	API p95 latency	Outcome
2025-09-17	1,281	1,281	248,904	v87	5.6 s	Normal
2025-09-18	1,284	913	177,420	v88	4.9 s	Bad release passed
2025-09-19	1,284	1,284	249,317	v89	5.8 s	Rebuilt and restored

The architecture sketch under review is compact but telling.



Questions the review panel wants answered

1. **[Analyze]** Which decision option best matches the primary consumer need without overreaching the team's implementation capacity, and which exhibit matters most in making that call?
2. **[Evaluate]** Given the source assessment table, which sources should be in scope for the first release, and what specific controls justify any "usable with controls" recommendation?
3. **[Analyze]** Does the Sep 18 incident argue for abandoning RAG, tightening release gates, or simplifying the architecture? Use at least two numbers from the exhibits.
4. **[Apply]** If you had to write one architecture decision record today, what would you lock down about storage pattern, transformation strategy, and serving interface?
5. **[Evaluate]** Which metrics belong in the daily operational dashboard, and which impressive-looking metrics would you deliberately leave out?

Model answers Priya wished the team had rehearsed

For question 1, the strongest answer is **decision option 1**. The technician is the primary consumer, and the contract requires concise answers with citations under **8 seconds p95**. Option 2 supports analysts but underserves technicians because retrieval-only search still forces them to read multiple documents. The most important exhibit is the consumer target table, not the architecture sketch. Consumer need should shape the architecture, not the other way around.

For question 2, first release should include work orders, parts inventory, manuals, and bulletins. Work orders and manuals are plainly usable. Parts inventory is **usable with controls** because of the **3.4% SKU alias mismatch**, which means normalization and quality checks belong in the cleaned layer before planner-facing use. Bulletins are **usable with controls** because late postings happen **2-3 times per month**; they need manifest completeness checks and release gating before a new index is promoted. Voice notes are deferred because **61% completeness** is too weak for trusted assistance, and warranty attachments are out of scope because high-sensitivity free text plus no owner SLA is a bad governance bargain.

For question 3, the Sep 18 incident argues for **tightening release gates**, not abandoning RAG. Retrieval precision@5 crashed from **0.79 to 0.54**, but the root cause is visible: only **913 of 1,284** bulletins landed and yet **v88** was still published. Latency even looked better at **4.9 seconds**, which is exactly why output-quality and completeness signals matter more than speed alone. A lakehouse-plus-RAG design remains valid if corpus, embedding, and index releases are version-linked and promotion is blocked on completeness and eval checks.

For question 4, the ADR should lock three things. First, **lakehouse-first zoning** with restricted raw, cleaned, curated, and AI-ready access layers because the platform must preserve raw documents and replayable artifacts. Second, **hybrid transformation**: Python and Spark for ingestion and text cleanup, dbt and SQL for the curated trusted product. Third, **FastAPI as the serving contract**, because an API makes inputs, outputs, latency targets, and error semantics explicit in a way notebooks and demo dashboards do not.

For question 5, the daily operational dashboard should stay narrow: pipeline success, freshness, manifest completeness, chunk-count deltas, retrieval precision@5 on the holdout set, grounded answer pass rate on the versioned eval suite, FastAPI p95 latency, and unresolved PII findings. Leave out vanity counts like total prompts, total chunks stored, or cumulative embeddings generated. Those numbers grow while trust can still collapse, as the Sep 18 release proved.

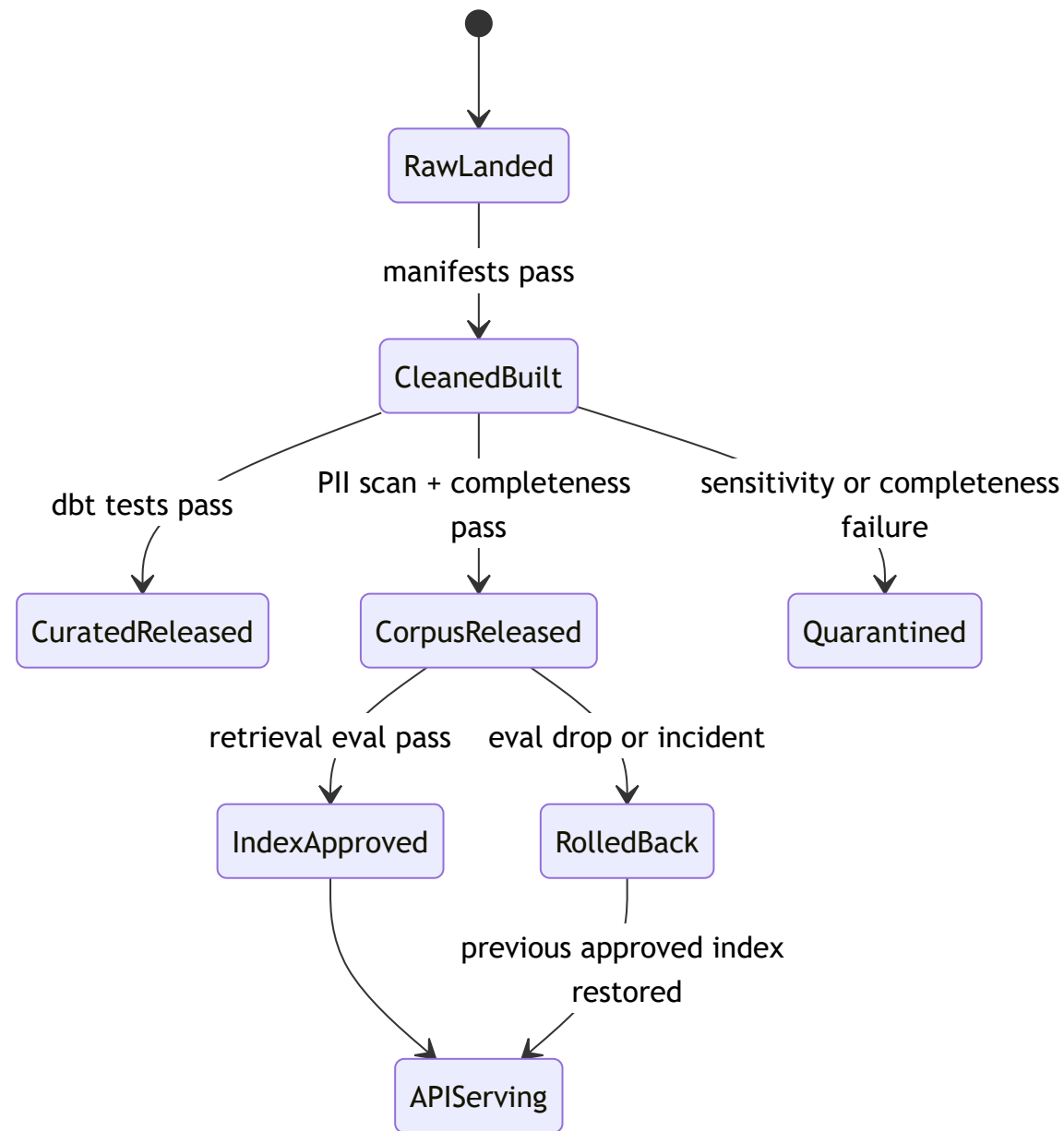
Warning: The panel will usually reward a smaller system with hard evidence over a larger system with soft claims. If your recommendation cannot name the owner, threshold, and evidence source for a metric, it is probably not operationally useful.

How the platform should run after the applause

The final review is not just about architecture; it is about whether MetroLift can operate the thing on Monday. Priya's runbook draft narrows recovery actions by artifact, not by panic level.

Failure signal	Likely broken artifact	First action	Escalate to	Restore trust with
Source row count drop > 10%	Raw landing or source access	Rerun ingest with manifest diff	Data engineer	New manifest + source-owner confirmation
Chunk count delta > 15% without approved change	Cleaned text or chunker	Rebuild corpus only	Data engineer	Chunk audit sample + lineage check
Retrieval precision@5 below 0.78	Embeddings or index	Roll back to prior approved index	ML engineer	Eval rerun + release note
PII finding in AI-ready text	Corpus filter gap	Block promotion and quarantine release	Governance owner	Scan report + signoff
dbt contract failure on curated service facts	Transformation logic	Fix and backfill curated tables	Analytics engineer	Passing tests + freshness restored

A state view made the team's recovery story finally click for reviewers.



Reflection prompt: Think of a platform you know that claims to be "AI-ready." Where would you draw the first hard scope boundary today, and which single metric would force the team to confront whether the build is helping its primary consumer or merely generating activity?

Transfer the lesson beyond MetroLift

- **Start** with a use case brief detailed enough that another engineer could infer consumers, outputs, risks, and likely architecture before a line of code is written.
- **Separate** business, AI, and platform metrics so a fast system with bad answers does not masquerade as a successful one.
- **Assess** each source as an operational contract: owner, freshness, sensitivity, quality, and recommendation are design inputs, not paperwork.
- **Prefer** the minimum viable AI-native component that directly serves the named consumer; here that is governed RAG, not a speculative agent.
- **Produce** evidence on every run through manifests, versioned releases, tests, and handoff notes so incidents can be replayed instead of guessed at.
- **Gate** releases according to the risks that actually break trust: incomplete corpora, PII leakage, retrieval drift, and silent schema changes.

Decision Points

Option 1. Build a lakehouse-first platform with restricted raw, cleaned, curated, and AI-ready access layers, then add a governed RAG path served through FastAPI.

- **Expected impact:** This should give MetroLift one platform that supports both the field assistant and analyst reporting while preserving raw evidence, replayability, and lineage.
- **Key risk:** It adds implementation surface area and will fail if the team cannot keep ingestion manifests, dbt contracts, and embedding releases disciplined.
- **Evidence to confirm:** Daily runs complete before 05:30 with versioned manifests, curated product freshness under 24 hours, and retrieval groundedness above the target on technician queries.
- **Evidence to kill:** If the team cannot produce reproducible corpus and index versions after a failed run, the lakehouse design is too ambitious for the project window.

Option 2. Keep the design warehouse-first and limit the build to structured ticket analytics plus document search, skipping generative answers.

- **Expected impact:** This reduces delivery risk and simplifies governance, but still delivers a useful curated product for planners and support analysts.
- **Key risk:** It may miss the primary consumer need for concise, evidence-backed answers in the field where technicians cannot scan long search results.
- **Evidence to confirm:** Technician task-completion trials show retrieval-only search answers the top 20 repair questions within the latency target and without higher escalation rates.
- **Evidence to kill:** If users repeatedly open multiple PDFs per task or ask follow-up clarifying questions, retrieval alone is not sufficient.

Option 3. Attempt a broader agent-style dispatch copilot that reads tickets, recommends parts, drafts customer updates, and triggers workflows.

- **Expected impact:** If it worked, it would create a headline-grabbing demo with visible automation value across operations.

- **Key risk:** This explodes scope, compounds governance risk, and turns a buildable capstone into a brittle orchestration problem with weak evidence boundaries.
- **Evidence to confirm:** Only proceed if every action path has a policy gate, approved source lineage, rollback plan, and simulation evidence covering error-prone cases.
- **Evidence to kill:** If core retrieval quality, source contracts, and incident runbooks are still unresolved, an agent path should be ruled out immediately.

Analysis Walkthrough

Start with the tempting wrong answer: option 3. MetroLift's VP of Service wanted "an assistant that fixes dispatch friction end to end," and the team almost translated that into an agent. The exhibits say not yet. Two of the five planned sources are unstable or sensitive: technician voice notes have inconsistent terminology and only 61% transcript completeness in the pilot week, while warranty claim attachments include customer phone numbers and free-text addresses. Building action-taking automation on top of that is how capstones become theater.

Now compare option 2 and option 1 on the actual consumer contract. The primary consumer is not an analyst. It is a field technician on a tablet at 07:10 before the first elevator call. The top business metric is repeat-visit rate on common faults, and the AI metric is answer groundedness with visible citations. In the table, technicians need under 8 seconds end-to-end, evidence on every answer, and daily freshness for service bulletins. Retrieval-only search helps analysts, but it offloads synthesis back to the technician. That conflicts with the stated consumer need.

The source assessment pushes the storage decision. MetroLift has structured work orders from SQL Server, semi-structured parts inventory extracts in CSV, and unstructured manuals and service bulletins in PDF. That is classic mixed-modality terrain. A warehouse-first design could hold curated ticket facts cleanly, but it is awkward as the base system of record for raw document landing, chunk lineage, and embedding rebuild evidence. The ClearWater and BlueRail examples from the Module 3 and Module 4 videos point toward a lakehouse-first layout because the platform has to preserve raw artifacts, cleaned text, curated facts, and AI-ready releases side by side.

Transformation should also be hybrid, not ideological. Use Python for ingestion and document normalization, Spark where the PDF

corpus and transcript cleanup need scalable text processing, and dbt plus SQL for curated trusted products consumed by planners and audit users. That split is justified by the numbers: 1.8 million historical work orders, 42,000 manuals and bulletins, and a daily delta of about 9,500 new or updated records. Pure ETL into final schemas would make change handling painful; pure ELT leaves too much ungoverned document logic floating outside a repeatable contract.

The incident exhibit is the tie-breaker. On 2025-09-18, retrieval precision@5 dropped from 0.79 to 0.54 after a partial bulletin ingest. The raw corpus had 1,284 expected files; only 913 were landed, but the embedding job still published a new index version because the pipeline lacked a manifest gate between corpus build and index release. That failure is not an argument against RAG. It is an argument for evidence-producing pipelines: restricted raw landing, run manifests, chunk-level lineage, release promotion rules, and a rollback path that can restore corpus v2025.09.17 and the prior index v87 together.

So I would pick option 1, with one explicit tradeoff: accept higher implementation complexity in exchange for a platform that can serve both AI and analytics credibly. The right-sized scope is narrower than it sounds. In scope: four approved sources, one curated trusted data product, one governed corpus, one RAG API in FastAPI, daily orchestration, and a documented recovery playbook. Deferred: voice-note ingestion into the production corpus, automated dispatch actions, and model fine-tuning.

The metrics should stay tight. Business: reduce repeat visits on the pilot fault classes from 18.4% to 15% in 90 days. AI: citation coverage 100%, grounded answer pass rate at least 85% on a 120-question eval set, retrieval precision@5 at least 0.78, p95 API latency under 8 seconds. Platform: daily pipeline success at 99%, curated freshness under 24 hours, failed-run recovery under 2 hours, and zero unauthorized access events. Anything broader becomes dashboard wallpaper.

Governance and CI/CD need risk-based gates, not generic best practices. Block release automatically if schema contracts fail, corpus completeness falls below 98%, PII detection on AI-ready text exceeds 0 findings unresolved, or retrieval eval drops more than 7 points from the prior approved release. Require manual review for new source onboarding, risk-note changes, and any release touching access policies. That matches the logic of the BeaconCart example from the Module 6 video: automate the mechanical checks, reserve human judgment for boundary changes.

If this were my team's final review, I would tell them the capstone earns credibility not by promising more intelligence, but by proving control. MetroLift does not need a magical copilot. It needs a reproducible platform where every answer can be traced back to an approved bulletin, every failed run can be replayed, and every scope decision is visible in writing.